CARMEL
Central School
Vellapad

PROJECT WORK IN

COMPUTER SCIENCE

(2026-2027)

Submitted by
Name :
Class :
Reg No :

BONAFIDE CERTIFICATE

Certified to be a bonafide record of the project work done by

Name: _____

Reg.No: _____

Class: _____

CARMEL CENTRAL SCHOOL VALAPAD

Principal

Teacher in charge

Submitted for

Senior Secondary School Examination 2026-2027

At Carmel Central School, Valapad

Thrissur, Kerala

External Examiner

Internal examiner

DECLARATION

I, ...

do hereby declare that the project entitled

.....

is a bonafide record of the project work done by me and was under the guidance of Ms. Soumya N. S.

I also declare that this project or any part of it has not been submitted by me fully or partially for any other examination before.

Place :

Date :

Signature :

ACKNOWLEDGEMENT

I am externally thankful to Almighty God and my parents who supported me throughout my work.

I express my sincere gratitude to Principal Sister Rincy for providing me with all facilities and also for the constant inspiration and encouragement for successful completion of this project.

I hereby express my abundant and heartfelt gratitude to Ms. Soumya N. S., my computer science teacher, who always gave valuable suggestions and guidance for completion of this project. They helped me to understand and remember important details of this project that I would have otherwise lost. My project has been a success only because of their guidance.

Signature of Student :
Name of Student :
Register Number :
Date :

SUPERMARKET BILLING SYSTEM



CONTENTS

1. Introduction .....

2. Manual .....

3. Synopsis .....

4. Source Code of the Project .....

5. GUI Interface .....

6. Bibliography .....

INTRODUCTION

This program is a simple supermarket billing system built using Python and the Tkinter library. It helps to manage the sales process efficiently by allowing the user to add items to a cart, calculate the total bill, and generate a bill with a unique bill number and date.

The system maintains an item database stored in a text file, making it easy to update and manage product details such as name, price, and unit. It also provides a user-friendly graphical interface for smooth and quick operation.

Overall, this application demonstrates the use of Python for real-world problem solving by creating an efficient, reliable, and easy-to-use billing system for a small supermarket.

ADVANTAGES

1. **Time Saving** – The system automates the billing process, reducing the time required to add items, calculate the total, and generate bills.

2. **Accuracy** – It minimizes human errors in price calculation and ensures correct totals for each purchase.

3. **Easy to Use** – The graphical user interface is simple and user-friendly, making it convenient for both customers and store staff.

4. **Efficient Record Management** – It maintains a digital database of items, prices, and sales records, which helps in quick updates and easy tracking of inventory and billing history.

MANUAL

This manual explains how to use the billing system, including its main features and functions, step by step.

1. Launch the Application

- Run the Python program.
- The main window of the billing system will open.

2. Add Items to Cart

- Enter the item name (as stored in the item database).
- Specify the quantity.
- Click "Add" to include the item in the cart.

3. View and Manage Cart

- Check the list of items in the cart.
- Remove items if needed.
- The total amount will be updated automatically.

4. Generate Bill

- Click the "Generate Bill" button.
- The bill number, date, item details, and total amount will be displayed.
- The bill can be printed or saved as required.

5. Manage Inventory

- Items and prices are stored in a text file.
- The system reads and updates the inventory automatically.
- New items can be added or existing items can be modified.

6. Exit the Program

- Close the window to end the program.

SYNOPSIS

The Supermarket Billing System is a simple and efficient application developed using Python and the Tkinter library. It is designed to help supermarket owners manage their sales process easily by allowing them to add items to a cart, calculate the total bill, and generate a bill with a unique bill number and date.

The program uses a text file to store the item details such as name, price, and unit. It also maintains an inventory of items and updates it automatically whenever a sale is made. The system provides a user-friendly graphical interface with options to add items, view the cart, remove items, and generate bills. It also ensures accurate calculation of the total amount and keeps track of the bill history.

Overall, this project demonstrates the use of Python and Tkinter in building a practical and easy-to-use billing system for a small supermarket. It helps in reducing manual work, improving accuracy, and making the billing process faster and more convenient.

PROGRAM SOURCE CODE

```
import tkinter as tk
from tkinter import ttk, messagebox
from datetime import datetime
import os
import math

# ================================================
# CONFIGURATION
# ================================================

ITEMS_FILE = "items.txt"
BILLNO_FILE = "billno.txt"

SHOP_NAME = "SIMON SUPERMARKET"
SHOP_LOCATION = "Valapad"
SHOP_PHONE = "9645541537"
SHOP_DETAILS = "Id 324567 ISI._ pvt.ltd"

BG = "#eef2f7"
WHITE = "#ffffff"
NAVY = "#1e3a5f"
BLUE = "#1976d2"
GREEN = "#16804a"
RED = "#c62828"
ORANGE = "#e67e22"
GRAY = "#64748b"


# ================================================
# FILE FUNCTIONS
# ================================================

def load_items():
    items = {}

    if not os.path.exists(ITEMS_FILE):
        return items

    try:
        with open(ITEMS_FILE, "r", encoding="utf-8") as file:
            for line in file:
                parts = line.strip().split(",")

                if len(parts) != 3:
                    continue

                try:
                    name = parts[0].strip().lower()
                    price = float(parts[1])
                    unit = parts[2].strip().lower()

                    if (
                        name
                        and "," not in name
                        and math.isfinite(price)
                        and price >= 0
                        and unit in ("kg", "piece")
                    ):
                        items[name] = (price, unit)

                except ValueError:
                    continue

    except OSError as error:
        messagebox.showerror("File Error", str(error))

    return items
```

```
def save_items(items):
    try:
        with open(ITEMS_FILE, "w", encoding="utf-8") as file:
            for name, (price, unit) in sorted(items.items()):
                file.write(f"{name},{price},{unit}\n")
        return True

    except OSError as error:
        messagebox.showerror(
            "Save Error",
            f"Could not save inventory:\n{error}"
        )
        return False


def get_bill_number():
    current = 1

    try:
        with open(BILLNO_FILE, "r", encoding="utf-8") as file:
            current = max(1, int(file.read().strip()))
    except (OSError, ValueError):
        current = 1

    try:
        with open(BILLNO_FILE, "w", encoding="utf-8") as file:
            file.write(str(current + 1))
    except OSError as error:
        messagebox.showerror(
            "Bill Number Error",
            f"Could not save the next bill number:\n{error}"
        )
        return None

    return current


# ================================================
# SCROLLABLE FRAME
# ================================================

class ScrollableFrame(ttk.Frame):
    """A frame whose contents can scroll vertically."""

    def __init__(self, parent, background=BG):
        super().__init__(parent)

        self.canvas = tk.Canvas(
            self,
            background=background,
            highlightthickness=0
        )

        self.scrollbar = ttk.Scrollbar(
            self,
            orient="vertical",
            command=self.canvas.yview
        )

        self.inner = tk.Frame(self.canvas, bg=background)

        self.window_id = self.canvas.create_window(
            (0, 0),
            window=self.inner,
            anchor="nw"
        )

        self.canvas.configure(yscrollcommand=self.scrollbar.set)
```

```
        self.canvas.pack(side="left", fill="both", expand=True)
        self.scrollbar.pack(side="right", fill="y")

        self.inner.bind("<Configure>", self.update_scroll_region)
        self.canvas.bind("<Configure>", self.resize_inner)

        self.canvas.bind_all("<MouseWheel>", self.mousewheel)
        self.canvas.bind_all("<Button-4>", self.linux_scroll_up)
        self.canvas.bind_all("<Button-5>", self.linux_scroll_down)

    def update_scroll_region(self, event=None):
        self.canvas.configure(
            scrollregion=self.canvas.bbox("all")
        )

    def resize_inner(self, event):
        self.canvas.itemconfigure(
            self.window_id,
            width=event.width
        )

    def mousewheel(self, event):
        if self.winfo_exists() and self.winfo_ismapped():
            self.canvas.yview_scroll(
                int(-event.delta / 120), "units"
            )

    def linux_scroll_up(self, event):
        if self.winfo_exists() and self.winfo_ismapped():
            self.canvas.yview_scroll(-1, "units")

    def linux_scroll_down(self, event):
        if self.winfo_exists() and self.winfo_ismapped():
            self.canvas.yview_scroll(1, "units")


# ================================================
# MAIN APPLICATION
# ================================================

class SupermarketBillingApp:

    def __init__(self, root):
        self.root = root
        self.root.title("Simon Supermarket Billing System")
        self.root.geometry("1100x760")
        self.root.minsize(760, 560)
        self.root.configure(bg=BG)

        self.items = load_items()
        self.cart = []
        self.last_bill_text = ""

        self.name_var = tk.StringVar()
        self.price_var = tk.StringVar()
        self.unit_var = tk.StringVar(value="kg")

        self.search_var = tk.StringVar()
        self.quantity_var = tk.StringVar()
        self.quantity_label_var = tk.StringVar(
            value="Quantity (grams)"
        )

        self.setup_styles()
        self.create_header()
        self.create_tabs()

        self.refresh_inventory()
        self.refresh_billing_items()
        self.refresh_cart()
```

```
        self.root.protocol("WM_DELETE_WINDOW", self.close_app)

    # ================================================
    # STYLES
    # ================================================

    def setup_styles(self):
        style = ttk.Style()

        try:
            style.theme_use("clam")
        except tk.TclError:
            pass

        style.configure(
            "Treeview",
            font=("Arial", 10),
            rowheight=28,
            background=WHITE,
            fieldbackground=WHITE
        )

        style.configure(
            "Treeview.Heading",
            font=("Arial", 10, "bold"),
            background="#dce6f1",
            foreground=NAVY
        )

        style.configure(
            "TNotebook",
            background=BG,
            tabmargins=(5, 5, 5, 0)
        )

        style.configure(
            "TNotebook.Tab",
            font=("Arial", 10, "bold"),
            padding=(14, 9)
        )

    def make_button(
        self, parent, text, command,
        color=BLUE, width=None
    ):
        options = {
            "text": text,
            "command": command,
            "bg": color,
            "fg": "white",
            "activebackground": color,
            "activeforeground": "white",
            "font": ("Arial", 10, "bold"),
            "relief": "flat",
            "cursor": "hand2",
            "padx": 10,
            "pady": 8,
            "wraplength": 180
        }

        if width is not None:
            options["width"] = width

        return tk.Button(parent, **options)

    def make_label(self, parent, text, **kwargs):
        defaults = {
            "text": text,
            "bg": WHITE,
```

```
            "fg": NAVY,
            "font": ("Arial", 10)
        }
        defaults.update(kwargs)
        return tk.Label(parent, **defaults)

    # ================================================
    # HEADER
    # ================================================

    def create_header(self):
        header = tk.Frame(self.root, bg=NAVY)
        header.pack(fill="x")

        tk.Label(
            header,
            text=SHOP_NAME,
            font=("Arial", 21, "bold"),
            fg="white",
            bg=NAVY
        ).pack(pady=(12, 3))

        tk.Label(
            header,
            text=(
                f"{SHOP_LOCATION}  |  Phone: {SHOP_PHONE}"
                f"  |  {SHOP_DETAILS}"
            ),
            font=("Arial", 9),
            fg="#e2e8f0",
            bg=NAVY
        ).pack(pady=(0, 12))

    # ================================================
    # TABS
    # ================================================

    def create_tabs(self):
        self.notebook = ttk.Notebook(self.root)
        self.notebook.pack(
            fill="both",
            expand=True,
            padx=10,
            pady=10
        )

        self.inventory_tab = ttk.Frame(self.notebook)
        self.billing_tab = ttk.Frame(self.notebook)
        self.output_tab = ttk.Frame(self.notebook)

        self.notebook.add(
            self.inventory_tab, text="  Inventory  "
        )
        self.notebook.add(
            self.billing_tab, text="  Billing  "
        )
        self.notebook.add(
            self.output_tab, text="  Final Bill  "
        )

        self.create_inventory_tab()
        self.create_billing_tab()
        self.create_output_tab()

    # ================================================
    # INVENTORY TAB
    # ================================================

    def create_inventory_tab(self):
        page = ScrollableFrame(self.inventory_tab)
```

```
        page.pack(fill="both", expand=True)

        content = page.inner

        tk.Label(
            content,
            text="INVENTORY MANAGEMENT",
            font=("Arial", 16, "bold"),
            bg=BG,
            fg=NAVY
        ).pack(anchor="w", padx=15, pady=(15, 10))

        form = tk.Frame(content, bg=WHITE, padx=15, pady=15)
        form.pack(fill="x", padx=15, pady=5)

        tk.Label(
            form, text="Item Name", bg=WHITE
        ).grid(row=0, column=0, sticky="w", padx=5)

        tk.Label(
            form, text="Price (Rs.)", bg=WHITE
        ).grid(row=0, column=1, sticky="w", padx=5)

        tk.Label(
            form, text="Unit", bg=WHITE
        ).grid(row=0, column=2, sticky="w", padx=5)

        self.name_entry = ttk.Entry(
            form, textvariable=self.name_var
        )
        self.name_entry.grid(
            row=1, column=0, sticky="ew", padx=5, pady=7
        )

        self.price_entry = ttk.Entry(
            form, textvariable=self.price_var
        )
        self.price_entry.grid(
            row=1, column=1, sticky="ew", padx=5, pady=7
        )

        self.unit_combo = ttk.Combobox(
            form,
            textvariable=self.unit_var,
            values=["kg", "piece"],
            state="readonly"
        )
        self.unit_combo.grid(
            row=1, column=2, sticky="ew", padx=5, pady=7
        )

        for column in range(3):
            form.columnconfigure(column, weight=1)

        actions = tk.Frame(form, bg=WHITE)
        actions.grid(
            row=2, column=0, columnspan=3,
            sticky="ew", pady=(8, 0)
        )

        for column in range(3):
            actions.columnconfigure(column, weight=1)

        self.make_button(
            actions, "Add / Update Item",
            self.add_item, GREEN
        ).grid(row=0, column=0, sticky="ew", padx=4, pady=4)

        self.make_button(
            actions, "Remove Selected",
```

```
        self.remove_item, RED
    ).grid(row=0, column=1, sticky="ew", padx=4, pady=4)

    self.make_button(
        actions, "Clear Fields",
        self.clear_item_fields, GRAY
    ).grid(row=0, column=2, sticky="ew", padx=4, pady=4)

    table_panel = tk.Frame(content, bg=WHITE)
    table_panel.pack(
        fill="both", expand=True, padx=15, pady=10
    )

    tk.Label(
        table_panel,
        text="AVAILABLE PRODUCTS",
        font=("Arial", 12, "bold"),
        fg=NAVY,
        bg=WHITE
    ).pack(anchor="w", padx=10, pady=10)

    tree_frame = tk.Frame(table_panel, bg=WHITE)
    tree_frame.pack(fill="both", expand=True, padx=8, pady=8)

    self.inventory_tree = ttk.Treeview(
        tree_frame,
        columns=("name", "price", "unit"),
        show="headings",
        selectmode="browse"
    )

    self.inventory_tree.heading("name", text="Item Name")
    self.inventory_tree.heading("price", text="Price (Rs.)")
    self.inventory_tree.heading("unit", text="Unit")

    self.inventory_tree.column(
        "name", width=220, anchor="w"
    )
    self.inventory_tree.column(
        "price", width=120, anchor="e"
    )
    self.inventory_tree.column(
        "unit", width=90, anchor="center"
    )

    inv_scroll = ttk.Scrollbar(
        tree_frame,
        orient="vertical",
        command=self.inventory_tree.yview
    )

    self.inventory_tree.configure(
        yscrollcommand=inv_scroll.set
    )

    self.inventory_tree.pack(
        side="left", fill="both", expand=True
    )
    inv_scroll.pack(side="right", fill="y")

    self.inventory_tree.bind(
        "<<TreeviewSelect>>",
        self.on_inventory_selected
    )

    self.make_button(
        content, "Refresh Inventory",
        self.refresh_inventory, BLUE
    ).pack(anchor="w", padx=15, pady=(0, 15))
```

```
    # ================================================
    # BILLING TAB
    # ================================================

    def create_billing_tab(self):
        page = ScrollableFrame(self.billing_tab)
        page.pack(fill="both", expand=True)

        content = page.inner

        tk.Label(
            content,
            text="BILLING COUNTER",
            font=("Arial", 16, "bold"),
            bg=BG,
            fg=NAVY
        ).pack(anchor="w", padx=12, pady=(12, 8))

        # Search products
        search_panel = tk.Frame(
            content, bg=WHITE, padx=12, pady=12
        )
        search_panel.pack(fill="x", padx=12, pady=5)

        tk.Label(
            search_panel,
            text="Search Item by Name",
            font=("Arial", 10, "bold"),
            bg=WHITE,
            fg=NAVY
        ).pack(anchor="w")

        search_row = tk.Frame(search_panel, bg=WHITE)
        search_row.pack(fill="x", pady=(5, 8))

        self.search_entry = ttk.Entry(
            search_row,
            textvariable=self.search_var,
            font=("Arial", 12)
        )
        self.search_entry.pack(
            side="left", fill="x", expand=True
        )

        self.search_entry.bind(
            "<KeyRelease>",
            self.refresh_billing_items
        )

        self.make_button(
            search_row, "Clear Search",
            self.clear_search, GRAY
        ).pack(side="left", padx=(8, 0))

        self.billing_count_label = tk.Label(
            search_panel,
            text="Select a product below",
            bg=WHITE,
            fg=GRAY,
            anchor="w"
        )
        self.billing_count_label.pack(fill="x")

        # Search results
        product_panel = tk.Frame(content, bg=WHITE, padx=10, pady=8)
        product_panel.pack(fill="x", padx=12, pady=5)

        tk.Label(
            product_panel,
            text="AVAILABLE PRODUCTS — SELECT ONE",
```

```
        font=("Arial", 11, "bold"),
        bg=WHITE,
        fg=NAVY
    ).pack(anchor="w", pady=(0, 6))

    product_frame = tk.Frame(product_panel, bg=WHITE)
    product_frame.pack(fill="x")

    self.billing_tree = ttk.Treeview(
        product_frame,
        columns=("name", "price", "unit"),
        show="headings",
        height=5,
        selectmode="browse"
    )

    self.billing_tree.heading("name", text="Item Name")
    self.billing_tree.heading("price", text="Price (Rs.)")
    self.billing_tree.heading("unit", text="Unit")

    self.billing_tree.column(
        "name", width=240, anchor="w"
    )
    self.billing_tree.column(
        "price", width=120, anchor="e"
    )
    self.billing_tree.column(
        "unit", width=90, anchor="center"
    )

    product_scroll = ttk.Scrollbar(
        product_frame,
        orient="vertical",
        command=self.billing_tree.yview
    )

    self.billing_tree.configure(
        yscrollcommand=product_scroll.set
    )

    self.billing_tree.pack(
        side="left", fill="x", expand=True
    )
    product_scroll.pack(side="right", fill="y")

    self.billing_tree.bind(
        "<<TreeviewSelect>>",
        self.on_billing_item_selected
    )
    self.billing_tree.bind(
        "<Double-1>",
        self.on_billing_item_double_click
    )

    # Quantity and add button
    quantity_panel = tk.Frame(
        content, bg=WHITE, padx=12, pady=12
    )
    quantity_panel.pack(fill="x", padx=12, pady=5)

    self.selected_item_label = tk.Label(
        quantity_panel,
        text="Selected item: None",
        bg=WHITE,
        fg=NAVY,
        font=("Arial", 11, "bold"),
        anchor="w"
    )
    self.selected_item_label.pack(fill="x", pady=(0, 8))
```

```
        qty_row = tk.Frame(quantity_panel, bg=WHITE)
        qty_row.pack(fill="x")

        self.make_label(
            qty_row, "Quantity (grams for kg items)"
        ).pack(anchor="w")

        self.quantity_label = tk.Label(
            qty_row,
            textvariable=self.quantity_label_var,
            bg=WHITE,
            fg=GRAY,
            font=("Arial", 9)
        )
        self.quantity_label.pack(anchor="w", pady=(2, 4))

        self.quantity_entry = ttk.Entry(
            qty_row,
            textvariable=self.quantity_var,
            font=("Arial", 12)
        )
        self.quantity_entry.pack(
            fill="x", pady=(0, 8)
        )

        self.quantity_entry.bind(
            "<Return>",
            lambda event: self.add_to_cart()
        )

        self.make_button(
            quantity_panel, "ADD SELECTED ITEM TO BILL",
            self.add_to_cart, GREEN
        ).pack(fill="x", pady=4)

        # Current cart
        cart_panel = tk.Frame(
            content, bg=WHITE, padx=10, pady=10
        )
        cart_panel.pack(fill="x", padx=12, pady=5)

        tk.Label(
            cart_panel,
            text="CURRENT BILL ITEMS",
            font=("Arial", 12, "bold"),
            bg=WHITE,
            fg=NAVY
        ).pack(anchor="w", pady=(0, 8))

        cart_frame = tk.Frame(cart_panel, bg=WHITE)
        cart_frame.pack(fill="x")

        self.cart_tree = ttk.Treeview(
            cart_frame,
            columns=("name", "qty", "unit", "rate", "amount"),
            show="headings",
            height=6,
            selectmode="browse"
        )

        headings = {
            "name": "Item",
            "qty": "Quantity",
            "unit": "Unit",
            "rate": "Rate (Rs.)",
            "amount": "Amount (Rs.)"
        }

        for col, title in headings.items():
            self.cart_tree.heading(col, text=title)
```

```
self.cart_tree.column("name", width=180, anchor="w")
self.cart_tree.column("qty", width=90, anchor="e")
self.cart_tree.column("unit", width=65, anchor="center")
self.cart_tree.column("rate", width=100, anchor="e")
self.cart_tree.column("amount", width=120, anchor="e")

cart_scroll = ttk.Scrollbar(
    cart_frame,
    orient="vertical",
    command=self.cart_tree.yview
)
self.cart_tree.configure(
    yscrollcommand=cart_scroll.set
)

self.cart_tree.pack(
    side="left", fill="x", expand=True
)
cart_scroll.pack(side="right", fill="y")

self.total_label = tk.Label(
    cart_panel,
    text="TOTAL: Rs. 0.00",
    bg=WHITE,
    fg=GREEN,
    font=("Arial", 16, "bold"),
    anchor="e"
)
self.total_label.pack(fill="x", pady=10)

# Buttons in a grid to keep them visible and organized
button_frame = tk.Frame(content, bg=BG)
button_frame.pack(fill="x", padx=12, pady=(5, 12))

for col in range(2):
    button_frame.columnconfigure(col, weight=1)

self.make_button(
    button_frame, "Remove Selected Item",
    self.remove_from_cart, ORANGE
).grid(
    row=0, column=0, sticky="ew", padx=4, pady=4
)

self.make_button(
    button_frame, "Clear All Bill Items",
    self.clear_cart, GRAY
).grid(
    row=0, column=1, sticky="ew", padx=4, pady=4
)

self.make_button(
    button_frame, "GENERATE FINAL BILL",
    self.generate_bill, GREEN
).grid(
    row=1, column=0, columnspan=2,
    sticky="ew", padx=4, pady=4
)

tk.Label(
    content,
    text=(
        "For kg items, enter grams. "
        "For piece items, enter whole numbers."
    ),
    bg=BG,
    fg=GRAY,
    font=("Arial", 9),
    wraplength=700
```

```
        ).pack(anchor="w", padx=15, pady=(0, 15))

    # ================================================
    # FINAL BILL OUTPUT TAB
    # ================================================

    def create_output_tab(self):
        panel = tk.Frame(self.output_tab, bg=WHITE)
        panel.pack(fill="both", expand=True, padx=10, pady=10)

        top = tk.Frame(panel, bg=WHITE)
        top.pack(fill="x", padx=10, pady=10)

        tk.Label(
            top,
            text="FINAL BILL / RECEIPT",
            font=("Arial", 15, "bold"),
            fg=NAVY,
            bg=WHITE
        ).pack(side="left")

        self.make_button(
            top, "Copy Bill",
            self.copy_bill, BLUE
        ).pack(side="right", padx=(5, 0))

        self.make_button(
            top, "Clear Display",
            self.clear_output, GRAY
        ).pack(side="right")

        self.bill_output = tk.Text(
            panel,
            wrap="word",
            font=("Courier New", 11),
            bg="#f8fafc",
            fg="#111827",
            padx=15,
            pady=15,
            spacing1=2,
            spacing3=2,
            undo=False
        )

        output_scroll = ttk.Scrollbar(
            panel,
            orient="vertical",
            command=self.bill_output.yview
        )

        self.bill_output.configure(
            yscrollcommand=output_scroll.set
        )

        output_scroll.pack(side="right", fill="y", padx=(0, 8), pady=8)
        self.bill_output.pack(
            side="left", fill="both", expand=True, padx=(8, 0), pady=8
        )

        self.bill_output.insert(
            "1.0",
            "SIMON SUPERMARKET\n"
            "Valapad\n\n"
            "Your completed bill will appear here.\n\n"
            "Open the Billing tab, search for an item, "
            "add products, and click GENERATE FINAL BILL."
        )
        self.bill_output.configure(state="disabled")

    # ================================================
```

```
    # INVENTORY OPERATIONS
    # ================================================

    def refresh_inventory(self):
        self.items = load_items()

        for row in self.inventory_tree.get_children():
            self.inventory_tree.delete(row)

        for name, (price, unit) in sorted(self.items.items()):
            self.inventory_tree.insert(
                "",
                "end",
                iid=name,
                values=(name, f"{price:.2f}", unit)
            )

        self.refresh_billing_items()

    def on_inventory_selected(self, event=None):
        selected = self.inventory_tree.selection()

        if not selected:
            return

        name = selected[0]

        if name not in self.items:
            return

        price, unit = self.items[name]

        self.name_var.set(name)
        self.price_var.set(str(price))
        self.unit_var.set(unit)

    def clear_item_fields(self):
        self.name_var.set("")
        self.price_var.set("")
        self.unit_var.set("kg")

        for row in self.inventory_tree.selection():
            self.inventory_tree.selection_remove(row)

    def add_item(self):
        name = self.name_var.get().strip().lower()
        price_text = self.price_var.get().strip()
        unit = self.unit_var.get().strip().lower()

        if not name:
            messagebox.showwarning(
                "Missing Name", "Enter the item name."
            )
            return

        if "," in name or "\n" in name:
            messagebox.showwarning(
                "Invalid Name",
                "Item names cannot contain commas or new lines."
            )
            return

        try:
            price = float(price_text)

            if not math.isfinite(price) or price < 0:
                raise ValueError

        except ValueError:
            messagebox.showerror(
```

```
                "Invalid Price",
                "Enter a valid, non-negative price."
            )
            return

        if unit not in ("kg", "piece"):
            messagebox.showwarning(
                "Invalid Unit", "Choose kg or piece."
            )
            return

        updated = self.items.copy()
        updated[name] = (price, unit)

        if save_items(updated):
            self.items = updated
            self.refresh_inventory()
            self.clear_item_fields()

            messagebox.showinfo(
                "Success",
                f"'{name}' saved successfully."
            )

    def remove_item(self):
        selected = self.inventory_tree.selection()
        name = selected[0] if selected else self.name_var.get().strip().lower()

        if not name or name not in self.items:
            messagebox.showwarning(
                "Select Item",
                "Select a valid item from the inventory."
            )
            return

        if any(item["name"] == name for item in self.cart):
            messagebox.showwarning(
                "Item In Current Bill",
                "Remove this item from the current bill before "
                "deleting it from inventory."
            )
            return

        if not messagebox.askyesno(
            "Confirm",
            f"Remove '{name}' from inventory?"
        ):
            return

        updated = self.items.copy()
        del updated[name]

        if save_items(updated):
            self.items = updated
            self.refresh_inventory()
            self.clear_item_fields()

            messagebox.showinfo(
                "Success", f"'{name}' removed."
            )

    # ================================================
    # BILLING SEARCH
    # ================================================

    def refresh_billing_items(self, event=None):
        if not hasattr(self, "billing_tree"):
            return

        query = self.search_var.get().strip().lower()
```

```
        for row in self.billing_tree.get_children():
            self.billing_tree.delete(row)

        matches = [
            name for name in sorted(self.items)
            if query in name
        ]

        for name in matches:
            price, unit = self.items[name]

            self.billing_tree.insert(
                "",
                "end",
                iid=name,
                values=(name, f"{price:.2f}", unit)
            )

        if hasattr(self, "billing_count_label"):
            self.billing_count_label.config(
                text=f"{len(matches)} product(s) found"
            )

    def clear_search(self):
        self.search_var.set("")
        self.refresh_billing_items()
        self.search_entry.focus_set()

    def on_billing_item_selected(self, event=None):
        selected = self.billing_tree.selection()

        if not selected:
            return

        name = selected[0]

        if name not in self.items:
            return

        price, unit = self.items[name]

        self.selected_item_label.config(
            text=f"Selected item: {name.title()}  |  "
                 f"Rs. {price:.2f} per {unit}"
        )

        if unit == "kg":
            self.quantity_label_var.set("Quantity (grams)")
        else:
            self.quantity_label_var.set("Quantity (pieces)")

        self.quantity_entry.focus_set()

    def on_billing_item_double_click(self, event=None):
        self.on_billing_item_selected()
        self.add_to_cart()

    # ================================================
    # CART OPERATIONS
    # ================================================

    def add_to_cart(self):
        selected = self.billing_tree.selection()

        if not selected:
            messagebox.showwarning(
                "Select Product",
                "Search for a product and select it first."
            )
```

```
            return

        name = selected[0]

        if name not in self.items:
            messagebox.showerror(
                "Product Missing",
                "This product is no longer in the inventory."
            )
            return

        try:
            quantity = float(self.quantity_var.get().strip())

            if (
                not math.isfinite(quantity)
                or quantity <= 0
                or quantity > 1_000_000_000
            ):
                raise ValueError

        except ValueError:
            messagebox.showerror(
                "Invalid Quantity",
                "Enter a valid quantity greater than zero."
            )
            return

        price, unit = self.items[name]

        if unit == "piece":
            if not quantity.is_integer():
                messagebox.showwarning(
                    "Invalid Quantity",
                    "Piece quantities must be whole numbers."
                )
                return

            quantity = int(quantity)
            amount = quantity * price
            display_qty = str(quantity)
            display_unit = "pcs"

        else:
            amount = quantity / 1000 * price
            display_qty = f"{quantity:g}"
            display_unit = "g"

        self.cart.append({
            "name": name,
            "quantity": quantity,
            "unit": unit,
            "price": price,
            "amount": amount,
            "display_qty": display_qty,
            "display_unit": display_unit
        })

        self.quantity_var.set("")
        self.refresh_cart()
        self.quantity_entry.focus_set()

    def refresh_cart(self):
        if not hasattr(self, "cart_tree"):
            return

        for row in self.cart_tree.get_children():
            self.cart_tree.delete(row)

        total = 0.0
```

```
        for index, item in enumerate(self.cart):
            total += item["amount"]

            self.cart_tree.insert(
                "",
                "end",
                iid=str(index),
                values=(
                    item["name"].title(),
                    item["display_qty"],
                    item["display_unit"],
                    f'{item["price"]:.2f}',
                    f'{item["amount"]:.2f}'
                )
            )

        self.total_label.config(
            text=f"TOTAL: Rs. {total:.2f}"
        )

    def remove_from_cart(self):
        selected = self.cart_tree.selection()

        if not selected:
            messagebox.showwarning(
                "Select Item",
                "Select an item in the current bill first."
            )
            return

        index = int(selected[0])

        if 0 <= index < len(self.cart):
            self.cart.pop(index)

        self.refresh_cart()

    def clear_cart(self):
        if not self.cart:
            messagebox.showinfo(
                "Empty Bill", "There are no items to clear."
            )
            return

        if messagebox.askyesno(
            "Clear Bill",
            "Remove all items from the current bill?"
        ):
            self.cart.clear()
            self.refresh_cart()

    # ================================================
    # GENERATE FINAL BILL
    # ================================================

    def generate_bill(self):
        if not self.cart:
            messagebox.showwarning(
                "Empty Bill",
                "Add products before generating the bill."
            )
            return

        if not messagebox.askyesno(
            "Confirm Bill",
            "Generate and finalize this bill?"
        ):
            return
```

```
        bill_no = get_bill_number()

        if bill_no is None:
            return

        now = datetime.now()
        total = sum(item["amount"] for item in self.cart)

        # Use shorter receipt lines to fit small screens.
        lines = [
            "=" * 38,
            SHOP_NAME,
            SHOP_LOCATION,
            f"Phone: {SHOP_PHONE}",
            SHOP_DETAILS,
            "=" * 38,
            "              SHOPPING BILL",
            f"Bill No: {bill_no}",
            f"Date: {now.strftime('%d-%m-%Y')}",
            f"Time: {now.strftime('%H:%M:%S')}",
            "-" * 38,
            "ITEM",
            "Qty       Rate (Rs.)    Amount (Rs.)",
            "-" * 38
        ]

        for item in self.cart:
            name = item["name"].title()
            qty = item["display_qty"] + " " + item["display_unit"]
            rate = item["price"]
            amount = item["amount"]

            # Long item names are wrapped rather than cut off.
            name_chunks = [
                name[i:i + 36] for i in range(0, len(name), 36)
            ]

            lines.append(name_chunks[0])

            for chunk in name_chunks[1:]:
                lines.append(chunk)

            lines.append(
                f"{qty:<10} {rate:>9.2f} {amount:>12.2f}"
            )

        lines.extend([
            "-" * 38,
            f"TOTAL AMOUNT: Rs. {total:.2f}",
            "=" * 38,
            "Thank You! Visit Again."
        ])

        bill_text = "\n".join(lines)

        self.last_bill_text = bill_text

        self.bill_output.configure(state="normal")
        self.bill_output.delete("1.0", tk.END)
        self.bill_output.insert("1.0", bill_text)
        self.bill_output.configure(state="disabled")
        self.bill_output.yview_moveto(0)

        self.cart.clear()
        self.refresh_cart()

        # Automatically show the completed bill.
        self.notebook.select(self.output_tab)

        messagebox.showinfo(
```

```
            "Bill Generated",
            f"Bill No: {bill_no}\n"
            f"Total: Rs. {total:.2f}\n\n"
            "The complete bill is displayed in the Final Bill tab."
        )

    # ================================================
    # BILL OUTPUT OPERATIONS
    # ================================================

    def copy_bill(self):
        text = self.bill_output.get("1.0", "end-1c").strip()

        if not text:
            messagebox.showwarning(
                "No Bill", "There is no bill to copy."
            )
            return

        self.root.clipboard_clear()
        self.root.clipboard_append(text)
        self.root.update()

        messagebox.showinfo(
            "Copied", "Bill text copied to the clipboard."
        )

    def clear_output(self):
        if not self.bill_output.get("1.0", "end-1c").strip():
            return

        if not messagebox.askyesno(
            "Clear Display",
            "Clear the displayed bill? This does not change inventory."
        ):
            return

        self.bill_output.configure(state="normal")
        self.bill_output.delete("1.0", tk.END)
        self.bill_output.configure(state="disabled")
        self.last_bill_text = ""

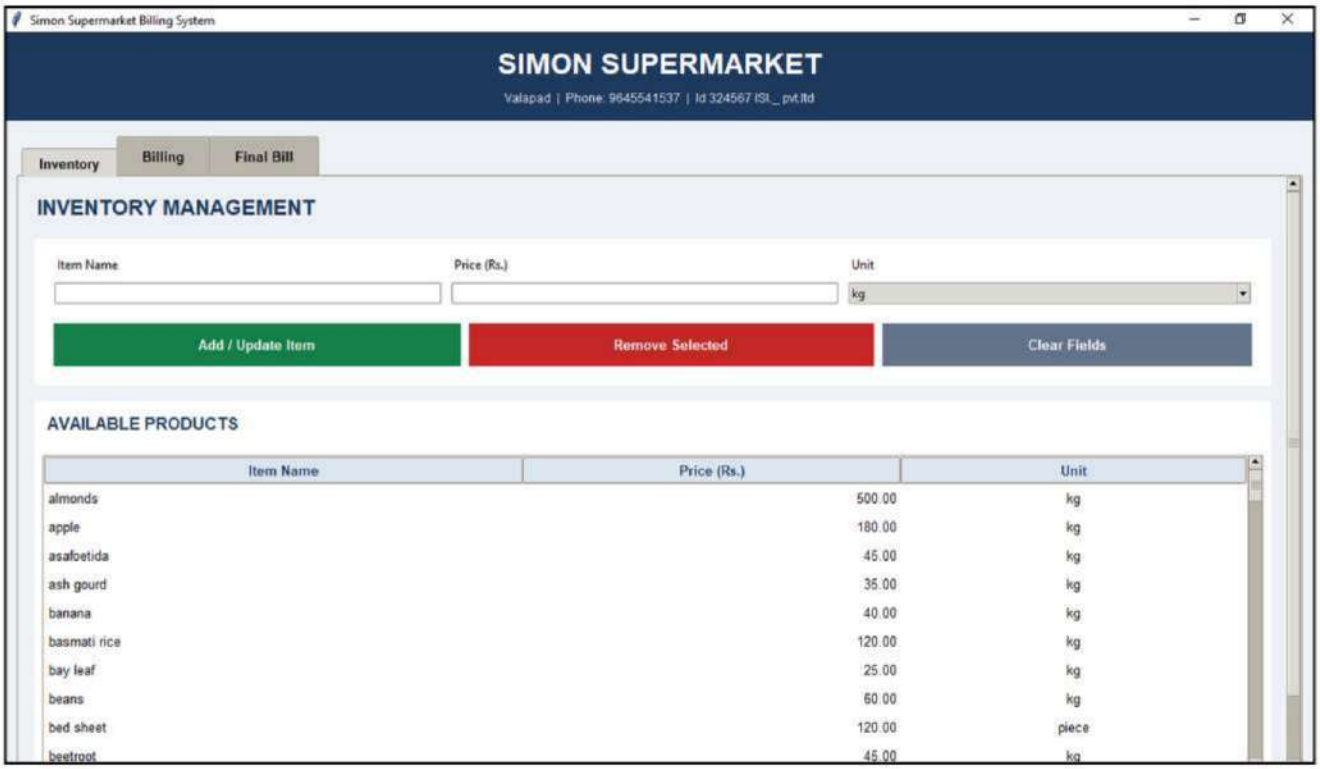
    # ================================================
    # CLOSE
    # ================================================

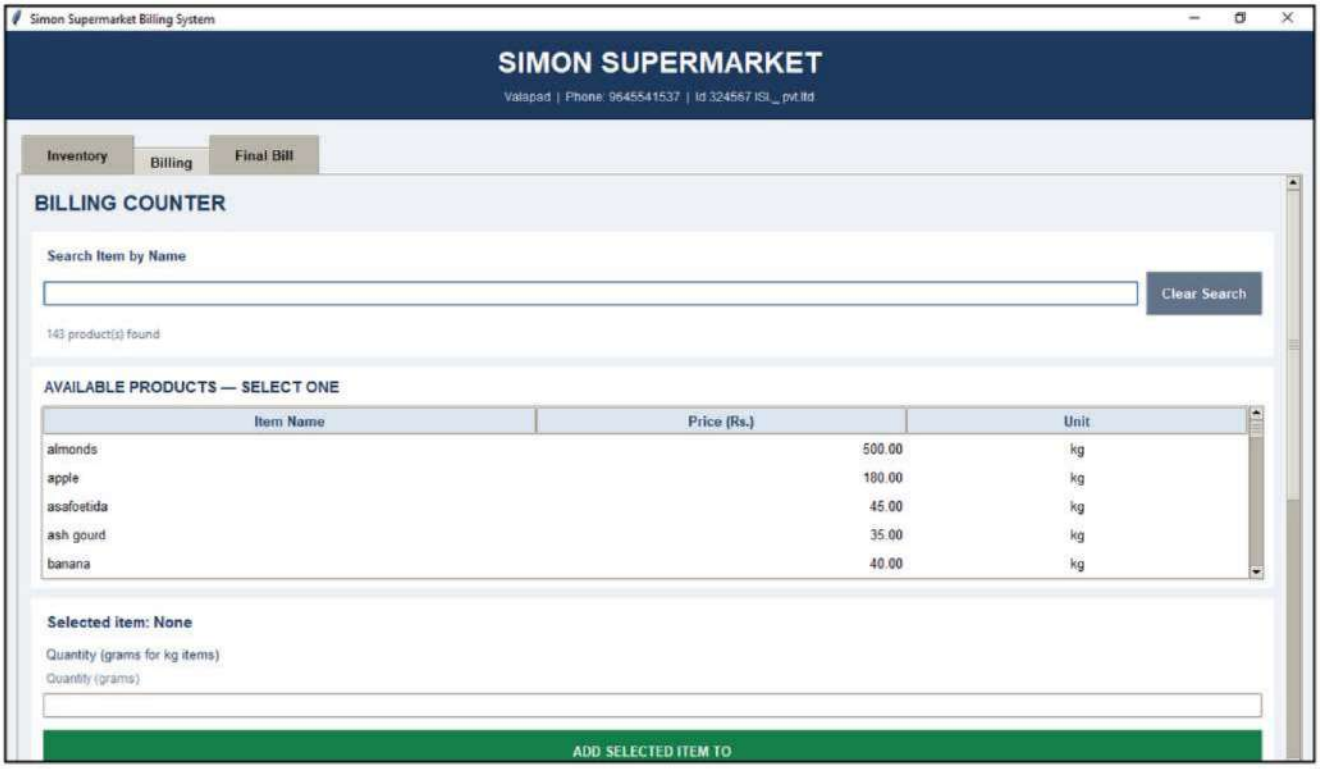
    def close_app(self):
        self.root.destroy()

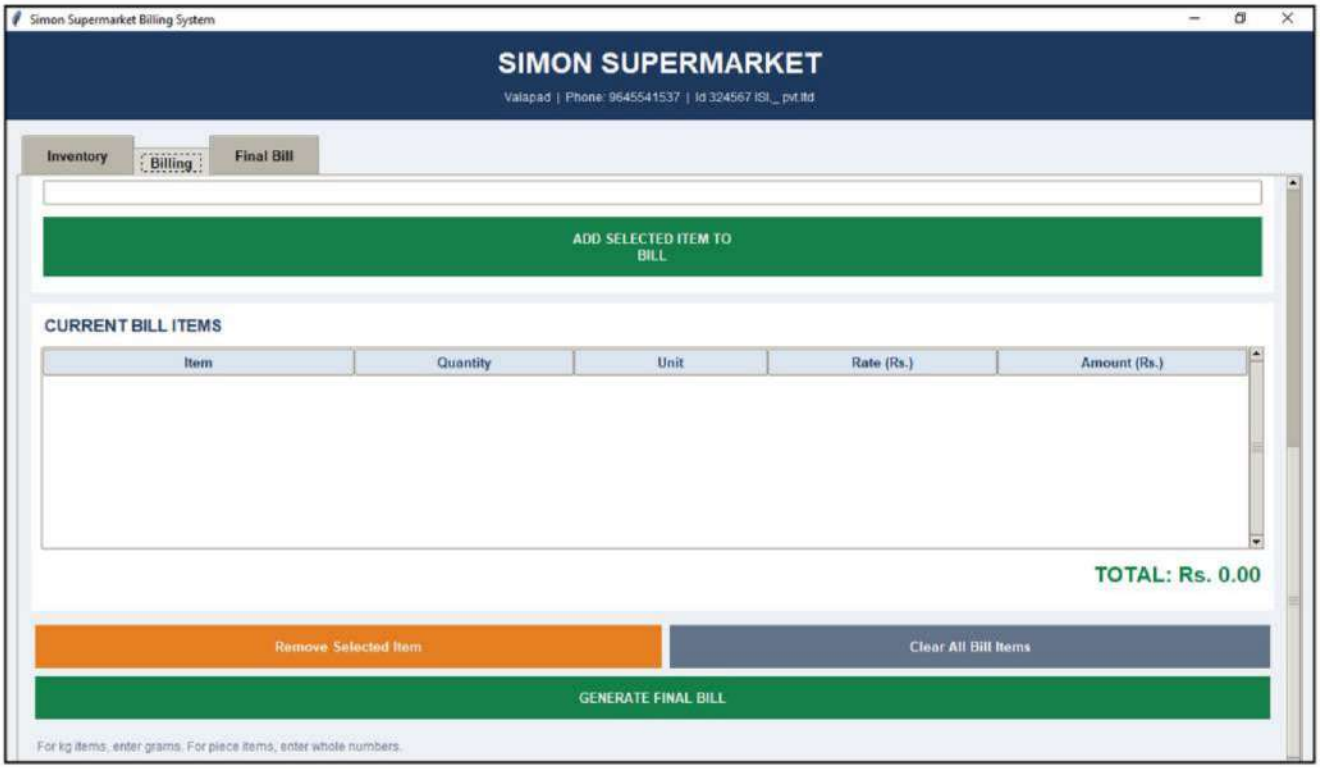

# ================================================
# RUN APPLICATION
# ================================================

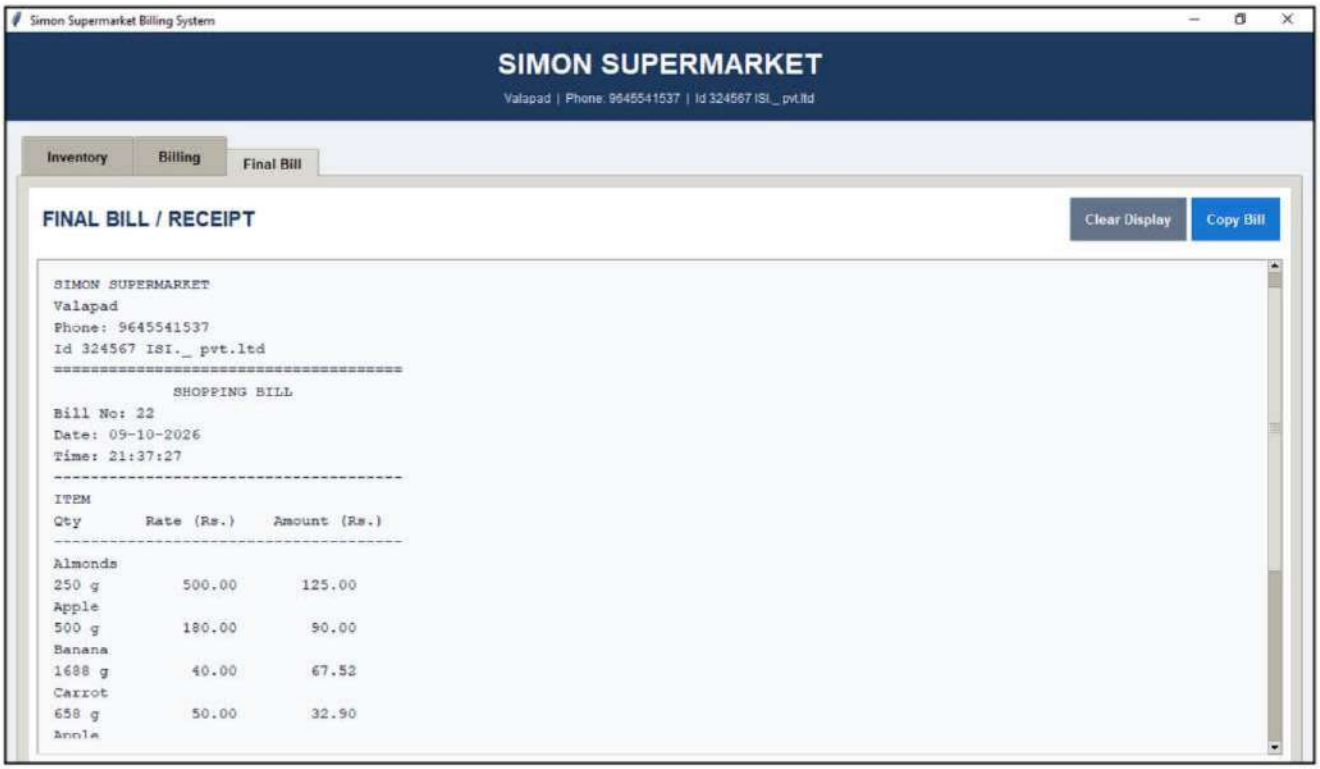
if __name__ == "__main__":
    root = tk.Tk()
    app = SupermarketBillingApp(root)
    root.mainloop()
```

SAMPLE OUTPUT






Simon Supermarket Billing System
SIMON SUPERMARKET
Valapad | Phone: 9645541537 | Id 324567 ISI_ pvt.ltd
Inventory
Billing
Final Bill
ADD SELECTED ITEM TO BILL
CURRENT BILL ITEMS
Item
Quantity
Unit
Rate (Rs.)
Amount (Rs.)
TOTAL: Rs. 0.00
Remove Selected Item
Clear All Bill Items
GENERATE FINAL BILL
For kg items, enter grams. For piece items, enter whole numbers.


Simon Supermarket Billing System
SIMON SUPERMARKET
Valapad | Phone: 9645541537 | Id 324567 ISI_ pvt.ltd
Inventory
Billing
Final Bill
FINAL BILL / RECEIPT
Clear Display
Copy Bill
SIMON SUPERMARKET
Valapad
Phone: 9645541537
Id 324567 ISI._ pvt.ltd
SHOPPING BILL
Bill No: 22
Date: 09-10-2026
Time: 21:37:27
ITEM
Qty Rate (Rs.) Amount (Rs.)
Almonds
250 g 500.00 125.00
Apple
500 g 180.00 90.00
Banana
1688 g 40.00 67.52
Carrot
658 g 50.00 32.90

BIBLIOGRAPHY

1. Python Documentation
   https://docs.python.org/3/

2. W3Schools – Python Tutorial
   https://www.w3schools.com/python/

3. GeeksforGeeks – Python Tkinter Tutorial
   https://www.geeksforgeeks.org/python/python-tkinter-tutorial/

4. TkDocs – Tkinter GUI Tutorial
   https://tkdocs.com/